

VMN TDMA Schedule Planner and Optimizer

Technical Documentation — Part 1

Technology: Python + NetworkX

Algorithm: Distance-2 Graph Coloring with Greedy Heuristics

Network Size: 16 Nodes | Communication Range: 500 meters

Prepared for VMN Technical Evaluation — Part 1

Table of Contents

1. Abstract
2. Problem Statement
3. Objectives
4. System Architecture
5. Technologies Used
6. Input Model
7. Communication Graph Construction
8. Distance-2 Conflict Graph
9. TDMA Scheduling as Graph Coloring
10. Scheduling Heuristic
11. Selected Strategy
12. Generated TDMA Schedule
13. TDMA Schedule Matrix
14. Spatial Reuse
15. Schedule Verification
16. Optimization Metrics
17. Algorithm
18. Pseudocode
19. Complexity
20. Why a Heuristic Was Used
21. Testing
22. Project Structure
23. Running the Project
24. Limitations
25. Future Enhancements
26. Engineering Design Decisions
27. Final Results
28. Conclusion
29. Repository
30. Declaration

1. Abstract

This project implements a centralized TDMA Schedule Planner and Optimizer for a wireless network of 16 nodes. The system accepts static node coordinates, constructs a communication graph using a 500-meter communication range, derives a distance-2 conflict graph to model direct and hidden-terminal interference, and generates a conflict-free TDMA schedule using graph-coloring heuristics. Three greedy coloring strategies are evaluated and the strategy producing the fewest slots is selected. For the provided topology, the implementation produced a valid 6-slot schedule with spatial reuse and automatically verified that no conflicting nodes transmit in the same slot.

2. Problem Statement

The objective is to assign TDMA transmission slots to network nodes while avoiding interference. Two nodes cannot transmit in the same slot when they are directly connected or when they are two hops apart through a common neighbor. Nodes that are sufficiently separated and do not conflict should be able to reuse the same slot. The optimizer must therefore reduce the total number of slots while maintaining a conflict-free schedule.

3. Objectives

- Build a communication graph from node coordinates using a 500-meter range.
- Model direct and two-hop interference using a distance-2 conflict graph.
- Generate a TDMA schedule using graph coloring.
- Compare multiple greedy coloring heuristics.
- Produce a Slot × Node binary schedule matrix and Node → Slot mapping.
- Verify the final schedule automatically.
- Report optimization metrics such as slot count and spatial reuse.

4. System Architecture

Node Coordinates → Input Validation → Communication Graph → Distance-2 Conflict Graph → Greedy Graph Coloring → Best Coloring Strategy → TDMA Schedule → Schedule Matrix + Node→Slot Mapping → Conflict Verification → Optimization Metrics

5. Technologies Used

Python, NetworkX, JSON input, command-line interface, and Python-based automated validation.

6. Input Model

The optimizer accepts a JSON object containing node names and two-dimensional coordinates. Example:

```
{"Node_01": [0, 0], "Node_02": [300, 0], "Node_03": [800, 0]}
```

The supplied project also contains the default 16-node topology required for the demonstration.

7. Communication Graph Construction

For every pair of nodes, Euclidean distance is calculated as:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

An edge is added when $d \leq 500$ meters. The resulting graph represents direct radio communication links. For the provided 16-node topology, the communication graph contains 31 links.

8. Distance-2 Conflict Graph

The conflict graph represents scheduling interference. A conflict edge is created when two nodes are directly connected or when they share a common neighbor. This models both direct-link interference and the two-hop/hidden-terminal condition specified by the problem. The resulting conflict graph contains 59 conflict links for the provided topology.

9. TDMA Scheduling as Graph Coloring

Graph coloring maps naturally to TDMA scheduling: each network node is a graph vertex, each conflict is an edge, and each color represents a TDMA slot. Adjacent vertices in the conflict graph must receive different colors. Therefore, a valid graph coloring produces a conflict-free TDMA schedule.

10. Scheduling Heuristic

The implementation evaluates three NetworkX greedy coloring strategies:

- largest_first
- smallest_last
- saturation_largest_first

Each strategy generates a complete coloring. The number of distinct colors/slots is measured, and the schedule with the fewest slots is selected.

11. Selected Strategy

For the provided 16-node topology, the selected strategy was largest_first. It produced a valid schedule using 6 TDMA slots. This is the best result among the evaluated greedy strategies; it is not claimed to be the mathematically optimal graph coloring.

12. Generated TDMA Schedule

Node-to-slot assignment:

Node_01 → Slot 1
Node_02 → Slot 3
Node_03 → Slot 4
Node_04 → Slot 5
Node_05 → Slot 6
Node_06 → Slot 1
Node_07 → Slot 2
Node_08 → Slot 3
Node_09 → Slot 4
Node_10 → Slot 5
Node_11 → Slot 6
Node_12 → Slot 1
Node_13 → Slot 3
Node_14 → Slot 2
Node_15 → Slot 5
Node_16 → Slot 4

13. TDMA Schedule Matrix

Column order: Node_01, Node_02, Node_03, Node_04, Node_05, Node_06, Node_07, Node_08, Node_09, Node_10, Node_11, Node_12, Node_13, Node_14, Node_15, Node_16

```
Slot 1: 1 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0
Slot 2: 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0
Slot 3: 0 1 0 0 0 0 0 1 0 0 0 0 1 0 0 0
Slot 4: 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1
Slot 5: 0 0 0 1 0 0 0 0 0 1 0 0 0 0 1 0
Slot 6: 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0
```

14. Spatial Reuse

Spatial reuse occurs because non-conflicting nodes can transmit during the same TDMA slot. The 16 nodes are scheduled across 6 slots, giving an average of 2.67 nodes per slot. For example, Slot 1 contains Node_01, Node_06, and Node_12 because those nodes do not conflict under the distance-2 model.

15. Schedule Verification

After coloring, every conflict edge is checked. If two conflicting nodes have the same slot, the schedule is rejected. The final schedule passed verification:

SUCCESS: No scheduling conflicts found.

16. Optimization Metrics

Total Nodes: 16

Communication Links: 31

Distance-2 Conflict Links: 59

TDMA Slots: 6

Selected Strategy: largest_first

Average Nodes per Slot: 2.67

Spatial Reuse: Enabled

Schedule Verification: Successful

17. Algorithm

1. Read or load node coordinates.
2. Validate the input structure and coordinate values.
3. Create a communication graph.
4. Add an edge for every node pair within 500 meters.
5. Create a conflict graph containing direct conflicts and common-neighbor conflicts.
6. Run multiple greedy graph-coloring strategies.
7. Count the slots generated by each strategy.
8. Select the strategy with the fewest slots.
9. Convert the coloring into a Slot × Node matrix.
10. Verify every conflict edge.
11. Print the schedule and optimization metrics.

18. Pseudocode

```
INPUT coordinates
VALIDATE coordinates
BUILD communication_graph
FOR each pair of nodes:
    IF distance <= 500:
        ADD communication edge
BUILD conflict_graph
FOR each node pair:
    IF directly connected OR share a neighbor:
        ADD conflict edge
FOR each greedy coloring strategy:
    COLOR conflict_graph
    COUNT required slots
SELECT coloring with minimum slots
BUILD schedule matrix
VERIFY every conflict edge
OUTPUT schedule, mapping, matrix, and metrics
```

19. Complexity

Communication graph construction checks node pairs and therefore has $O(N^2)$ pairwise distance work. Conflict-graph construction depends on graph density and neighborhood relationships. Greedy coloring is practical for the target network size, but the general minimum graph-coloring problem is computationally hard. The implementation therefore uses practical heuristics rather than exhaustive search.

20. Why a Heuristic Was Used

Finding the absolute minimum number of colors for a general graph is an NP-hard optimization problem. For a scheduling tool intended to work quickly, greedy coloring provides a practical solution. Evaluating multiple ordering strategies improves the chance of obtaining a compact schedule without requiring expensive exhaustive search.

21. Testing

Default topology test:

```
python scheduler.py
```

Result: 16 nodes, 31 communication links, 59 conflict links, 6 slots, average 2.67 nodes per slot, verification successful.

External JSON test:

```
python scheduler.py input.json
```

Result: external topology loaded and a valid schedule generated.

Automated test:

```
python test_scheduler.py
```

Result: TEST PASSED; 5 nodes; 6 communication links; 10 conflict links; 5 slots; largest_first.

22. Project Structure

```
VMN-TDMA-Optimizer/  
■■■ scheduler.py  
■■■ input.json  
■■■ test_scheduler.py  
■■■ README.md  
■■■ .gitignore
```

23. Running the Project

Install the dependency:

```
pip install networkx
```

Run the default topology:

```
python scheduler.py
```

Run with a JSON topology:

```
python scheduler.py input.json
```

Run automated validation:

```
python test_scheduler.py
```

24. Limitations

- Static network topology is used.
- Scheduling is centralized.
- Greedy optimization does not guarantee the global minimum number of slots.
- The interference model is graph-based rather than a physical-layer propagation model.
- Dynamic traffic, mobility, and changing link conditions are not currently modeled.
- The current project focuses on Part 1 and does not yet integrate EMANE.

25. Future Enhancements

Potential improvements include mobility support, dynamic topology updates, traffic-aware scheduling, node priorities, weighted graph coloring, more advanced optimization methods, visualization, real-time schedule updates, external TDMA configuration export, and future EMANE integration.

26. Engineering Design Decisions

A 500-meter distance threshold was used to construct the communication graph. Communication and conflict graphs were kept separate so that network connectivity and scheduling interference remain clear. A distance-2 conflict model was used to capture both direct and two-hop interference. Multiple greedy strategies were evaluated automatically, and the final schedule is always verified before being reported.

27. Final Results

The implementation successfully generated a conflict-free TDMA schedule for the 16-node topology. The selected greedy strategy was largest_first and the resulting schedule required 6 slots. Spatial reuse was achieved by assigning multiple non-conflicting nodes to the same slot. The schedule was automatically verified with no conflicts detected.

28. Conclusion

The Part 1 implementation provides a complete centralized TDMA scheduling pipeline: coordinate validation, communication graph construction, distance-2 conflict modeling, heuristic graph coloring, schedule matrix generation, conflict verification, and optimization metrics. The result is a practical and explainable scheduler that can serve as the foundation for future simulator integration.

29. Repository

GitHub repository: <https://github.com/Aslesha584/VMN-TDMA-Optimizer>

30. Declaration

AI-assisted development tools were used during implementation and documentation. The final algorithm, source code, tests, and reported results were reviewed and validated locally. The implementation is documented so that its design decisions and scheduling process can be explained during technical discussion.